Mechatronics Engineering (MCTR601)

Project Final Report

NIMBLE: Computer Vision Based Ball Balancing & Trajectory Tracking Robot

Name	ID	Lab Number
Zeina Abdelaal	61-9589	T-36
Ahmed Shawki	61-32383	T-35
Farah Abdelaal	61-9586	T-35

Supervised By:
Prof. Ayman A. El-Badawy

Table of Contents

Project Description

Overview

The project is a vision-based ball balancing robot that keeps a ball at the center of a 2-DOF tilting platform using camera feedback and closed-loop control. The system combines computer vision, embedded control, inverse kinematics and mechanical design to continuously detect the ball's position and adjust the platform angle to stabilize it. The challenge requires sensing the ball position in real time, computing corrective tilt angles, and actuating them fast enough to reject disturbances and maintain balance.

In addition to stabilization, the system also supports trajectory tracking, where the ball can be guided along predefined paths, such as a circular/infinity sign trajectory or a custom freehand sketch. This is achieved by dynamically updating the target position used to calculate the error, allowing the robot to not only balance the ball but also control its motion smoothly across the platform.

To enhance user interaction, the system includes a graphical user interface (GUI) for mode selection and trajectory drawing. It allows switching between different operating modes in real time, including center balancing mode, predefined trajectory tracking mode, and custom freehand drawing mode. Also enabling the user to sketch and edit the custom paths that the ball will follow on the platform.

How the System Works

A webcam captures images of the platform and detects the position of the ball (x,y) using image processing techniques (OpenCV running on a PC). The ball's x and y positional errors relative to the desired target setpoint are then computed and sent to the STM32 through serial communication.

The STM32 runs two independent PID control algorithms that calculate how much the platform should tilt in the x and y directions to move the ball toward the target position. Based on this calculation, the microcontroller commands two stepper motors that adjust the platform angle.

The platform is supported using a central mount and 2 robotic arms that are built using custom 3D-printed links and mechanical components, creating a stable structure that allows controlled tilting in two axes.

Through this continuous loop — camera detection → position calculation → PID control → motor adjustment — the robot maintains the ball balanced at the center of the platform in real time.

Components Used

Sensor:
The primary sensor is the overhead USB webcam, which provides the (x, y) position of the ball through OpenCV-based image processing on the host PC.

Actuators:
Two Nema 17 stepper motors driven by their dedicated motor drivers (A4988); each motor controls tilt about one of the two platform axes (X & Y).

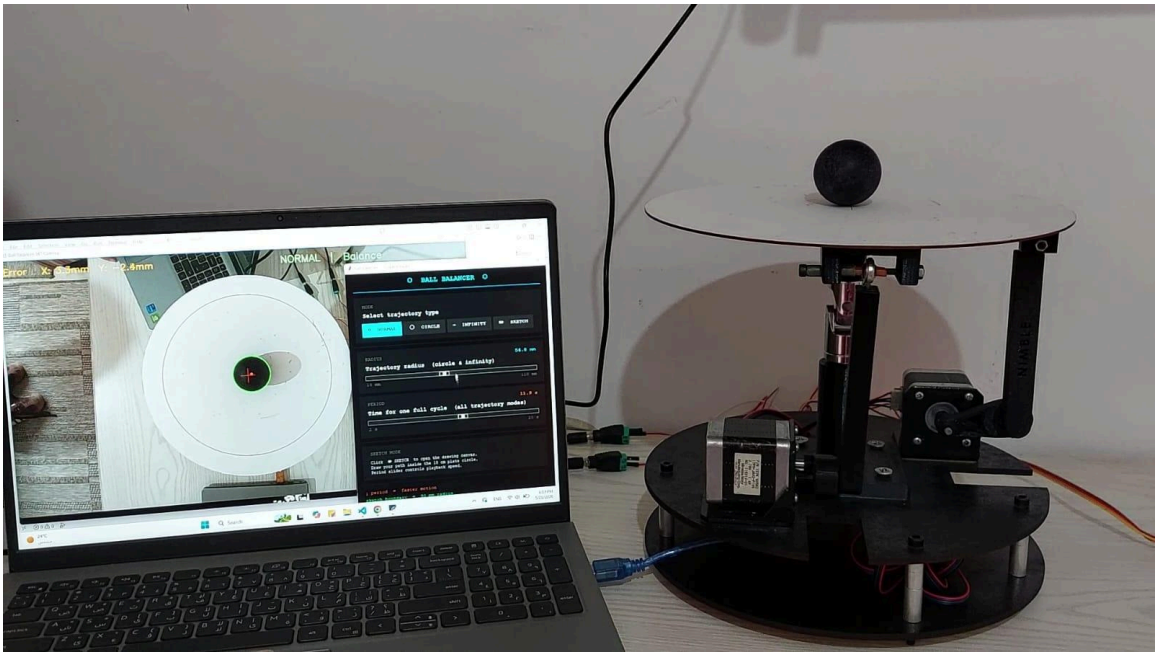
Microcontroller:

STM32 Blue Pill, acts as the real-time control unit, receives and parses the ball's positional error transmitted over USART, executes the PID loops and generates the required stepper step/direction signals accordingly.

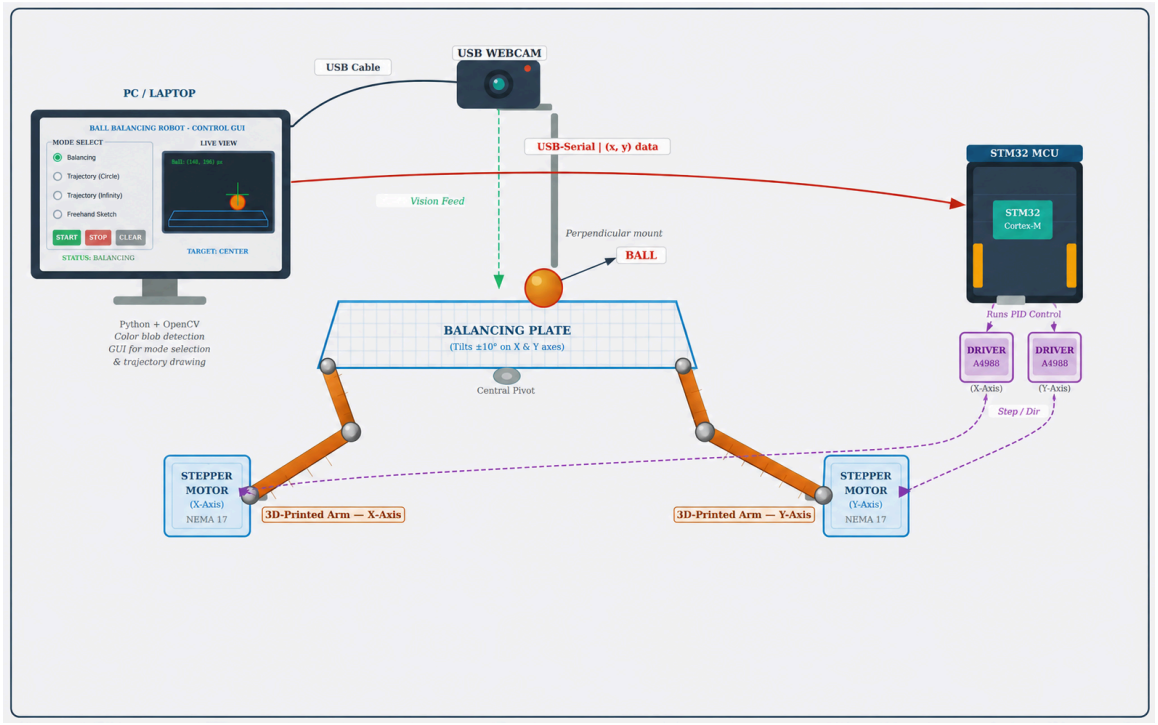
Communication Protocol:
A USB-to-TTL converter is used as the communication interface between the computer and the STM32. It transmits the ball's positional error (x,y), over serial communication (USART) to the STM32.

Power Supply:

Two 12V 2A adaptors to power both motors.



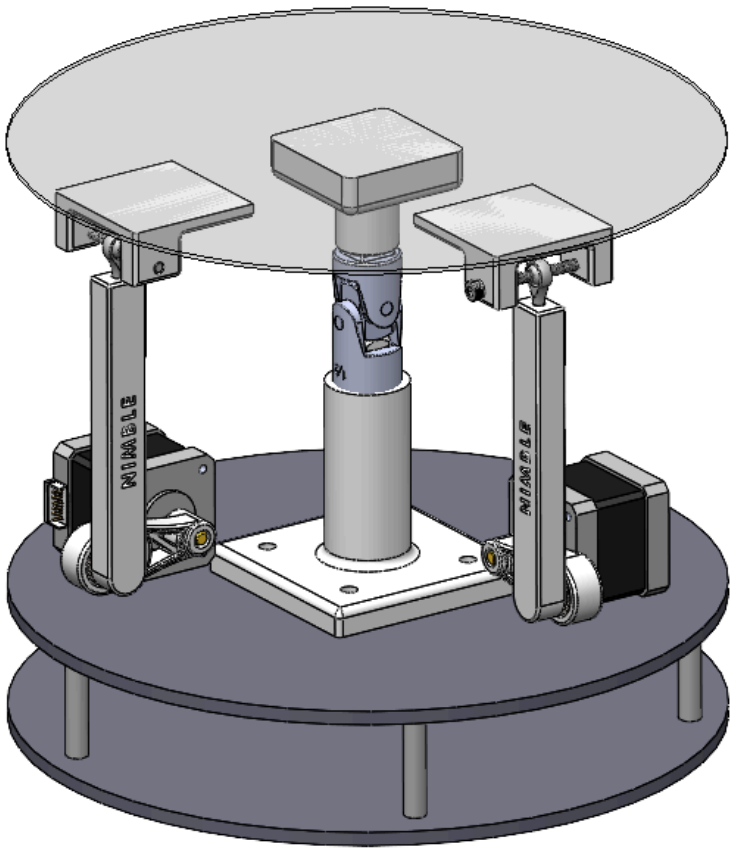
System Setup



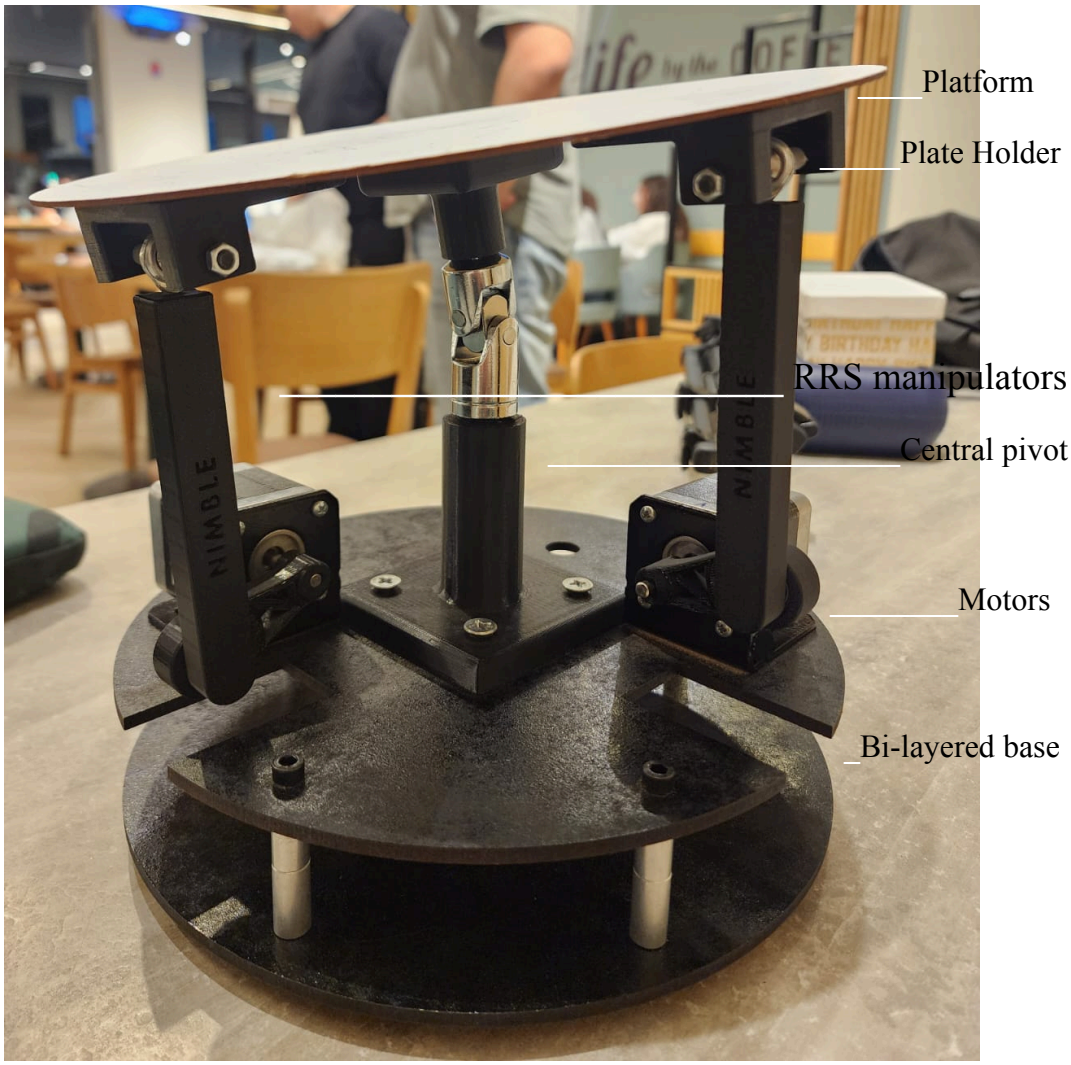
Functional Diagram

Methodology

1. Mechanical design



Robot Solidworks Assembly



Real life Hardware with key components labelled

RRS Parallel Manipulators Design

We designed two RRS (Revolute-Revolute-Spherical) manipulators, each driven by a stepper motor to control platform tilt along one axis independently.

Each leg consists of two rigid links, a short arm and a long arm, connected in series by three joints

Starting from the base, the short arm is press-fitted directly onto the stepper motor shaft, forming the first actuated revolute joint - a clean rotational connection driven by the motor itself.

The long arm is then connected to the short arm via a 688ZZ bearing, forming the second revolute joint and allowing the long arm to rotate relative to the short arm about a single fixed axis.

At the top of the long arm, an M4 spherical joint connects to the plate holder, forming the terminal spherical joint. This allows rotation about multiple axes, accommodating the platform's combined tilt from both legs simultaneously. The plate holder is fastened via an M4 bolt, anchoring the platform while permitting free orientation in response to the manipulators' motion.

Together, the two decoupled RRS legs produce a 2-DOF tilting platform, with each leg independently controlling tilt about one axis.

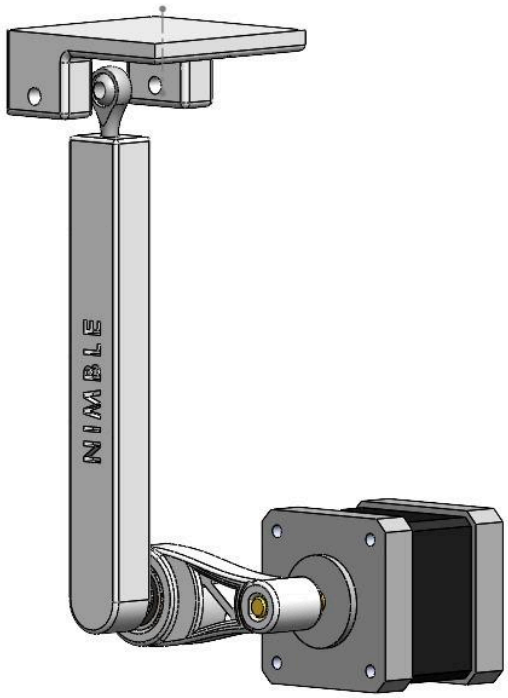


Figure 1: RRS parallel manipulator assembly

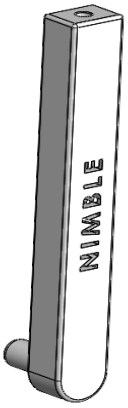


Figure 2: Long Arm

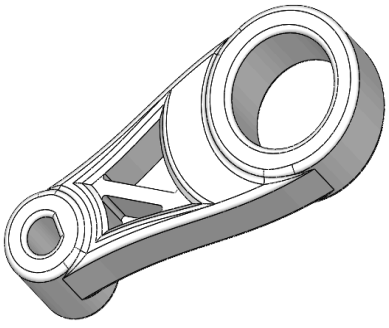


Figure 3: Short Arm

Figure 4: Spherical Joint

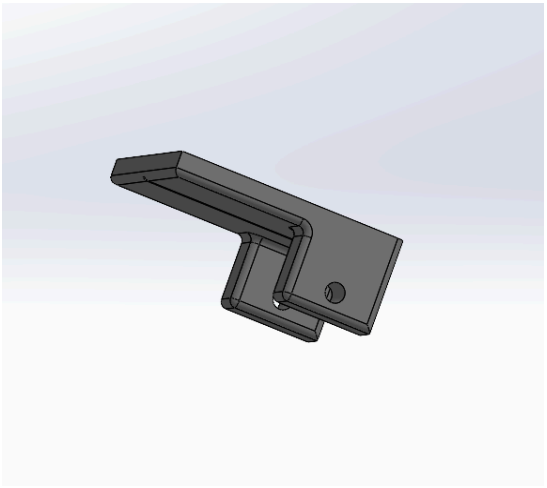
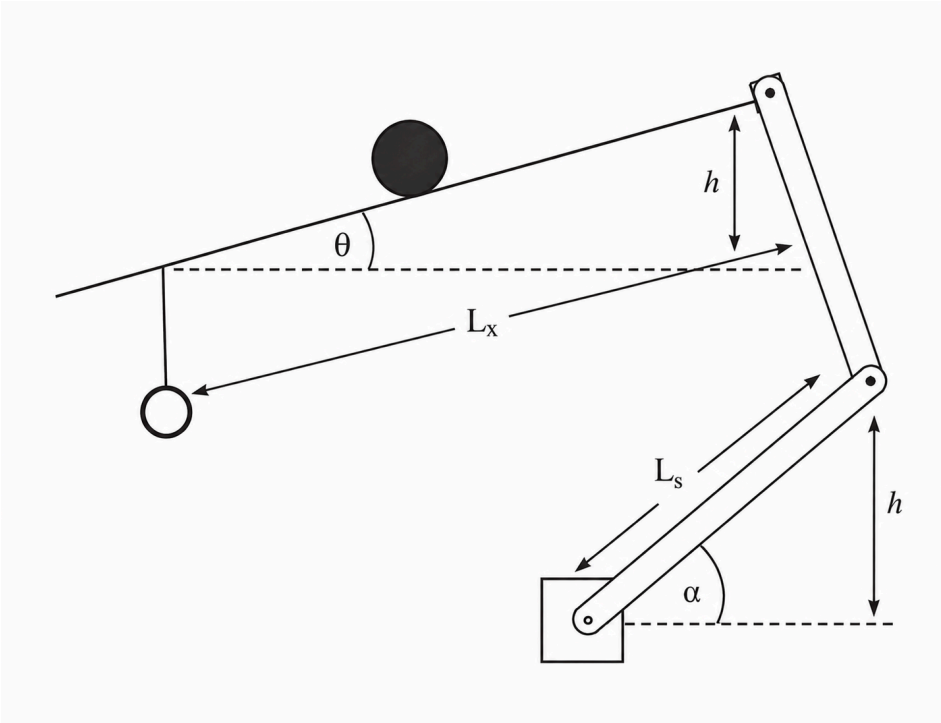


Figure 5: Plate Holder

Inverse Kinematics

To achieve a desired platform tilt angle, it was necessary to determine the number of motor steps required to produce the corresponding plate inclination.

The derivation of the relationship between the platform tilt angle **θ** and the motor rotation angle **α** is presented as follows:



For simplicity and due to the small range of motion the long arm is assumed to be vertical throughout the motion. It therefore transmits vertical displacement directly and without distortion from the crank tip to the plate attachment point. This gives the central assumption:

$$h_{\text{short arm}} = h_{\text{plate}}$$

Left side — vertical height of the short arm tip above the motor pivot:

$$h_{\text{short arm}} = Ls \sin(\boldsymbol{\alpha})$$

Right side — vertical height of the plate attachment point, at distance Lx along the plate tilted by **θ**:

$$h_{\text{plate}} = Lx \sin(\boldsymbol{\theta})$$

Setting equal:

$$Ls \sin(\boldsymbol{\alpha}) = Lx \sin(\boldsymbol{\theta})$$

Solving directly for the motor angle:

$$\boldsymbol{\alpha} = \arcsin (Lx/Ls) \sin(\boldsymbol{\theta})$$

Linearized Form (Small Angle Approximation)

applying $\sin\theta \approx \theta$ and $\sin\alpha \approx \alpha$:

$$\boldsymbol{\alpha} \approx Lx/Ls \, \boldsymbol{\theta}$$

This reveals a **constant linear gain** (Lx/Ls) between plate angle and motor angle.

The two axes are fully decoupled — **αx** depends only on **θx** and **αy** depends only on **θy**

The linearized model is suitable for control design in the neighbourhood of the home (flat) position, which is the primary operating region of the manipulator.

For our parameters Lx=10.5cm, Ls=4cm, and ⅛ microstepping, the microsteps per degree of plate tilt is derived as follows.

A standard stepper motor has 200 full steps per revolution. With 1/8 microstepping this gives:

$$N=200\times8=1600 \text{ microsteps/revolution}$$

From the linearized IK, the mechanical gain is:

$$\boldsymbol{\alpha}/\boldsymbol{\theta}=Lx/Ls=10.5/4=2.625$$

meaning each degree of plate tilt demands 2.625° of motor rotation.The number of microsteps per degree of plate tilt is therefore:

$$\text{microsteps per degree of plate tilt}=(1600 * 2.625)/360 = \mathbf{11.667 \text{ microsteps/}^\circ}$$

Mount

To support the platform, we designed a central mount that attaches directly to the platform's center. The mount is split into two parts — a top and a bottom — with a Universal Joint sitting between them.

The Universal Joint acts as the core of the mount, linking the two halves while allowing the plate to rotate freely in any direction, giving it full 360° of rotational freedom.

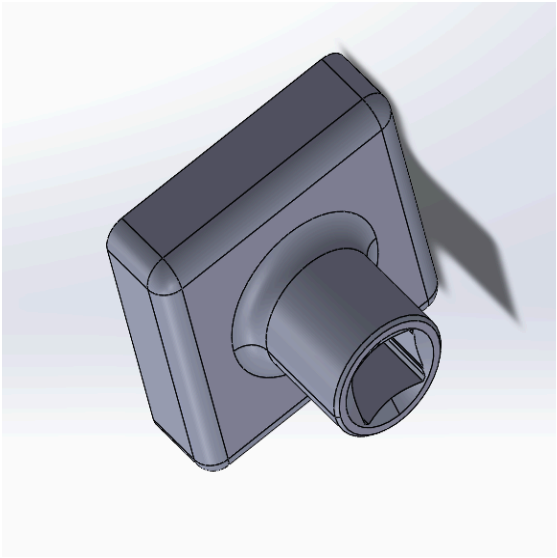


Figure 6: Top Mount

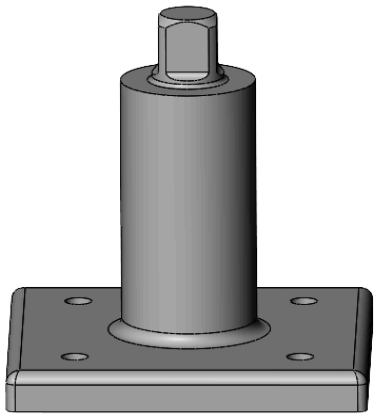


Figure 7: Bottom Mount

Figure 8: Universal Joint

Bi-Layered Base

The base consists of two 5mm MDF plates, laser cut for precision and separated by M5 spacers to create a structured two-tier frame.

The top platform houses the stepper motors and the central mount, keeping all mechanical components together in one layer. Cutouts are integrated into the platform, giving the arms the clearance they need to move freely without obstruction.

The bottom platform sits beneath, dedicated to the electrical components, keeping the wiring and electronics neatly separated from the mechanics above.

Platform

The platform was laser cut from 1.5 mm MDF with a diameter of 25 cm , with engravings on its underside marking the precise locations of the central pivot and plate holders. This ensured accurate positioning during assembly.

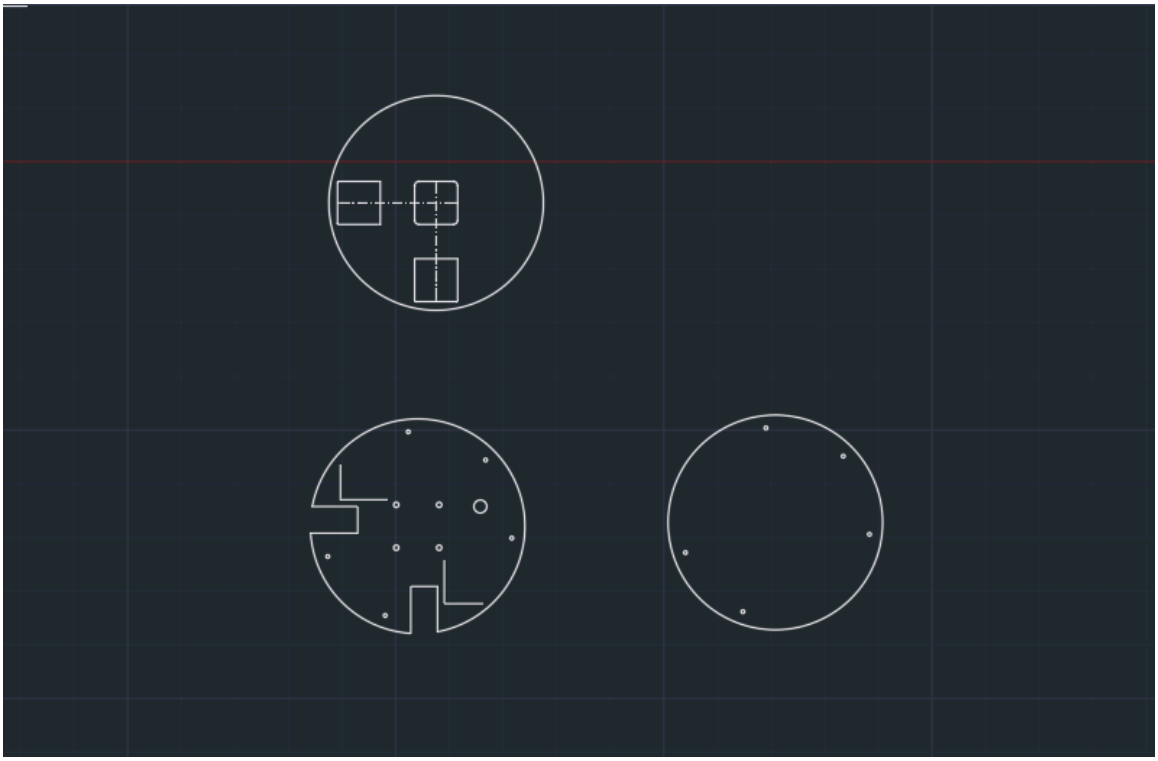
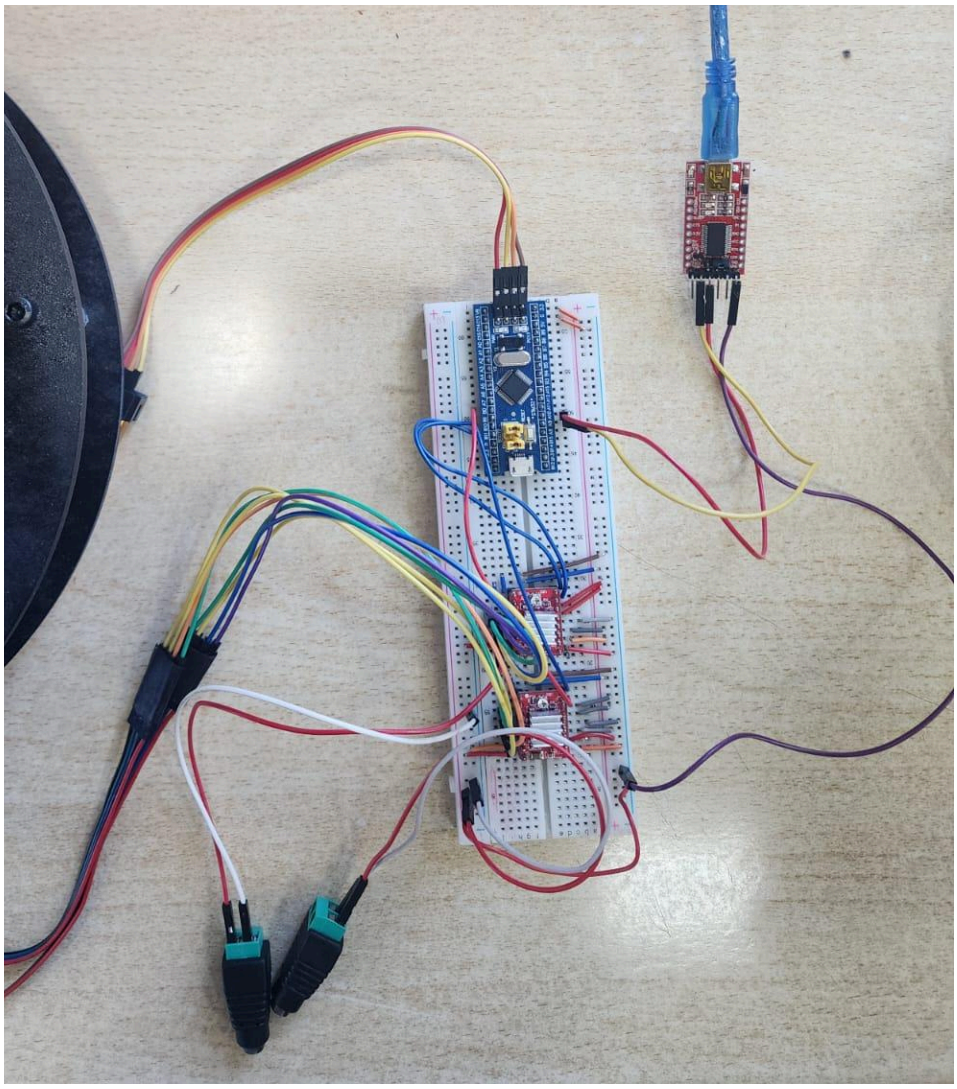


Figure 9: AutoCAD drawings for laser cutting

2.Electrical design



Components:

- SM32 Blue Pill & ST Link
- USB TTL Converter (To send X,Y positional errors to STM)
- Two A4988 Motor Drivers Connected to two Steppers
- Two 12V adapters (one for each stepper)

Control

1.Modeling

1.1 Governing Equations — Derivation and Linearization

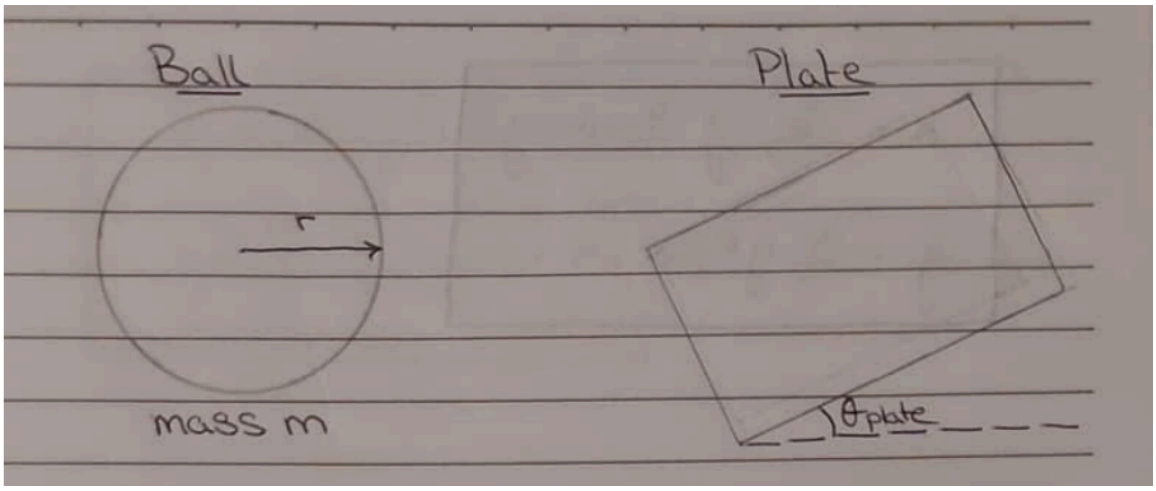
The ball balancing system is modelled by analysing the motion of a hollow thin-walled sphere rolling on a tilted plate. The following assumptions are applied:

Assumptions taken:

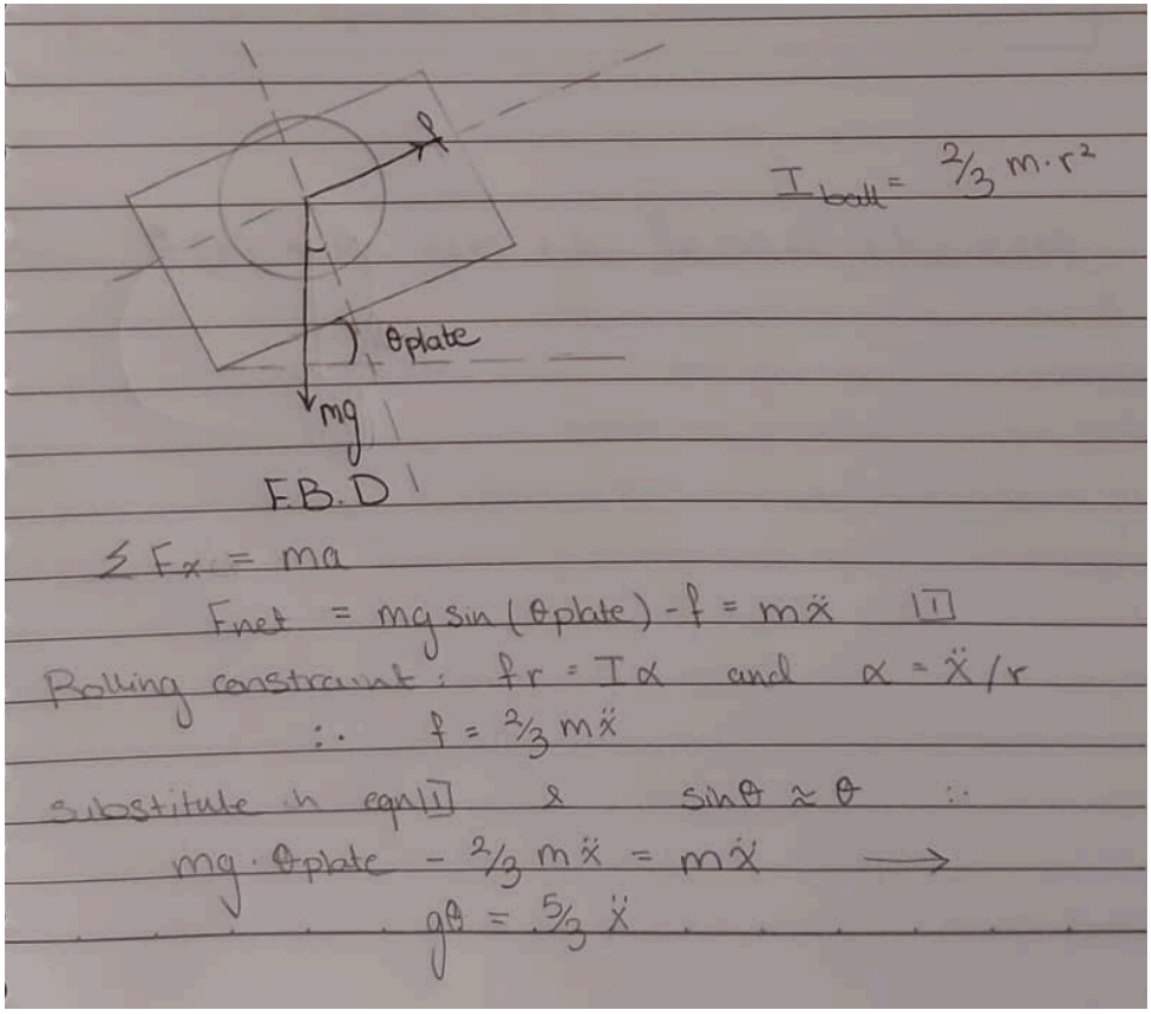
- The ball rolls without slipping on the plate surface
- Small angle approximation applies: $\sin(\theta) \approx \theta$ for tilt angles $\leq 8°$
- The plate is rigid, flat, and uniform
- Air resistance and rolling friction are neglected
- The two axes (X and Y) are mechanically decoupled and modelled independently
- The ball is a standard ping-pong ball modelled as a hollow thin-walled sphere with moment of inertia $I = (2/3)mr^2$

Equations of motion:

Consider the ball of mass m and radius r rolling on the plate, which is tilted at angle **θplate** about one axis.



Applying Newton's second law along the plate surface and the rotational equation about the ball's centre:



$I_{ball} = \frac{2}{3} m \cdot r^2$

$\sum F_x = ma$

$F_{net} = mg \sin(\theta_{plate}) - f = m\ddot{x}$ [1]

Rolling constraint: $fr = I\alpha$ and $\alpha = \ddot{x}/r$

$\therefore \quad f = \frac{2}{3} m\ddot{x}$

Substitute in eqn[1] & $\sin\theta \approx \theta$ $\therefore$

$mg \cdot \theta_{plate} - \frac{2}{3} m\ddot{x} = m\ddot{x} \longrightarrow$

$g\theta = \frac{5}{3} \ddot{x}$

$$\ddot{x} = \frac{3}{5} g \cdot \theta_{plate} - y$$

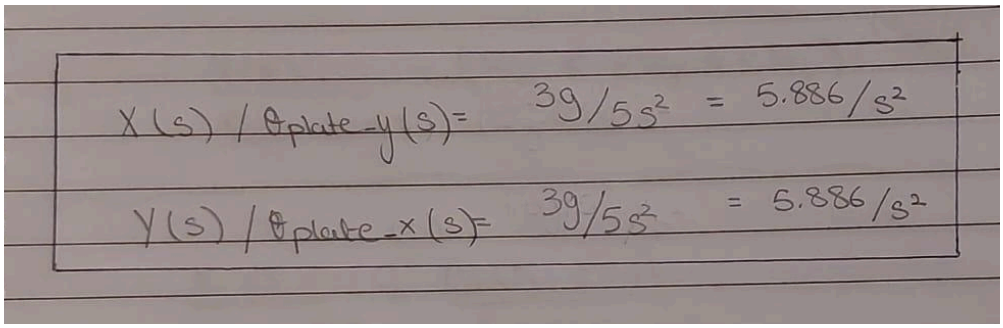
$$\ddot{y} = \frac{3}{5} g \cdot \theta_{plate} - x$$

where **g = 9.81 m/s²** is gravitational acceleration, **θplate_x and θplate_y** are the plate tilt angles about the X and Y axes respectively, and (3/5) is the rolling inertia factor for a hollow thin-walled sphere. Note that this gives a plant gain of K = (3/5)g = 5.886 m/s²/rad.

1.2 Transfer Function Model

Open Loop Transfer Function

Taking the Laplace transform of the equation of motion for a single axis, treating **θ_plate** as the control input and ball position x as the output:



$$G(s) = X(s) / \Theta_{plate}(s) = 5.886 / s^2$$

This is a **double integrator plant**. It is open-loop unstable with two poles at the origin, which is why active feedback control is required. Both axes share the same transfer function structure, so identical controllers are designed for each axis independently.

The stepper motor is modelled as a first-order lag with time constant τ = 0.0012 s (derived from the hardware step interval of 1200 µs from the C code):

$$M(s) = 1 / (0.0012s + 1)$$

The combined plant seen by the PID controller is:

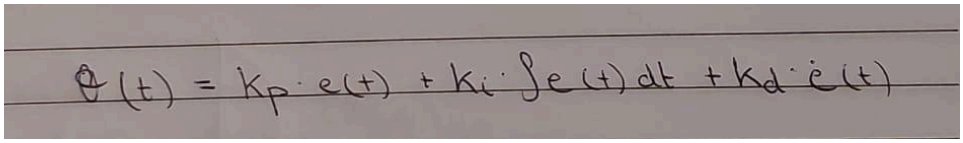
$$P(s) = M(s) \cdot G(s) = 5.886 / [s^2 \cdot (0.0012s + 1)]$$

The open-loop transfer function is:

```
L_OL(s) = P(s) = 5.886 / [s² · (0.0012s + 1)]
```

PID Control Law

Each axis is controlled by an independent PID controller of the form:



where:

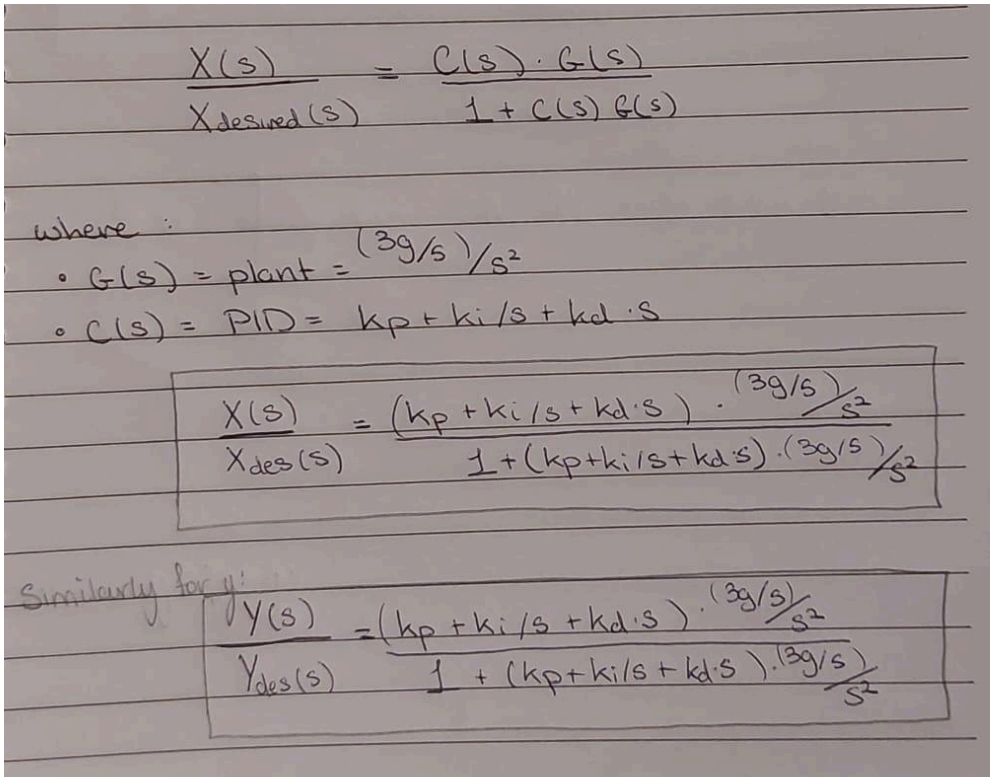
- e(t) = x_desired − x(t) for the X-axis, or e(t) = y_desired − y(t) for the Y-axis
- θ(t) is the commanded plate tilt angle sent to the stepper motor
- The output is saturated at ±8° matching the physical hardware limit from the STM32 C code

In transfer function form the PID controller is:

```
C(s) = Kp + Ki/s + Kd·s
```

Closed Loop Transfer Function

With the PID wrapped around it



Replace G(s) in the photo with P(s) (the combined plant)

The closed-loop transfer function is:

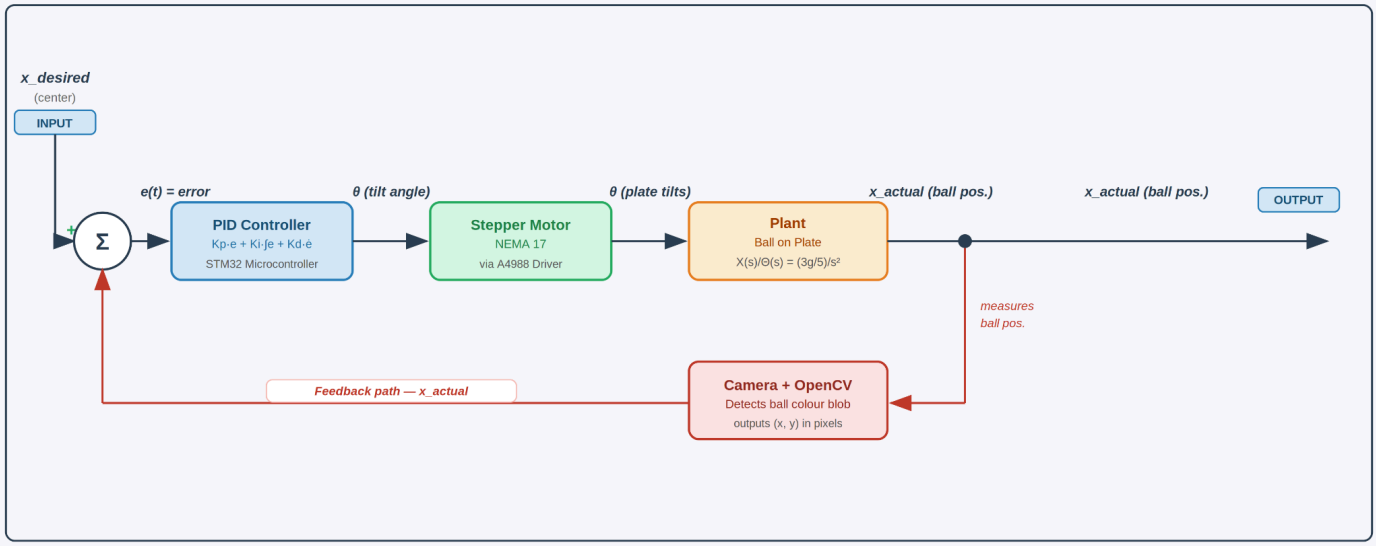
$$T(s) = [C(s) \cdot P(s)] / [1 + C(s) \cdot P(s)]$$

$$T(s) = \frac{[Kp + Ki/s + Kd \cdot s] \cdot 5.886 / [s^2 \cdot (0.0012s + 1)]}{1 + [Kp + Ki/s + Kd \cdot s] \cdot 5.886 / [s^2 \cdot (0.0012s + 1)]}$$

1.3 Block Diagram Representation

The closed-loop system for one axis is represented as follows:

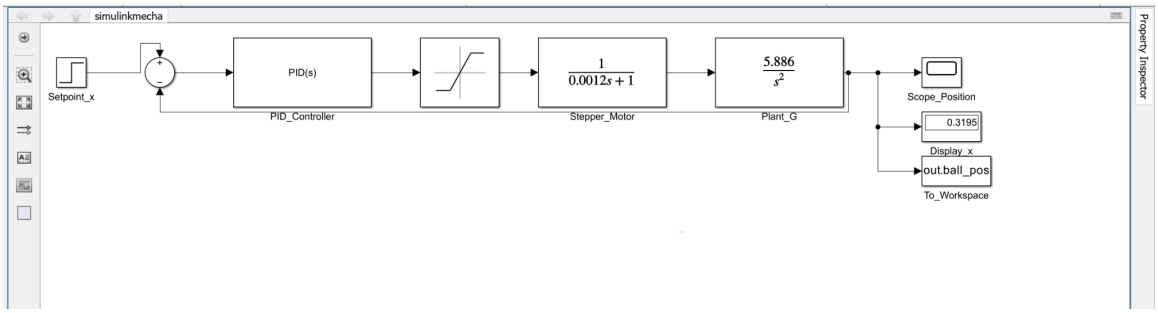
Closed-loop Block Diagram



*An identical independent loop runs for the Y-axis.

Simulink Model:

The complete system was also implemented in Simulink with the following blocks connected in series:



Block	Transfer Function	Description
Setpoint	x_desired	Desired ball position (center = 0, but we made at simulation 0.3 to visualize it more)
Summing Junction	Σ (+/-)	Computes error: e(t) = x_desired - x(t)
PID Controller	C(s) = PIDF	Computes tilt angle command
Saturation	±8 degrees	Limits output to physical hardware range
Stepper Motor	1 / (0.0012s + 1)	First-order lag model
Plant	5.886 / s^2	Ball rolling dynamics
Feedback	Camera + OpenCV	Measures ball (x,y) position in pixels

2. Analysis — Open Loop Response

Open-loop poles:

Pole	Location	Meaning
p_1, p_2	s = 0 (double)	Double integrator — open-loop unstable
p_3	s = −833	Stepper motor lag

Open-loop analysis conclusions:

- **Two poles at the origin** confirm the system is open-loop unstable — any small tilt causes the ball to accelerate indefinitely away from center
- The Bode plot shows a −40 dB/decade slope at low frequencies, characteristic of a double integrator
- The open-loop step response shows ball position diverging to infinity, confirming that **active closed-loop control is mandatory**

3. Controller Design

3.1 Controller Design Steps

A PID controller (PID with derivative filter) was selected for the following reasons:

- The double integrator plant requires derivative action to introduce damping
- The integral term eliminates steady-state error as the ball settles at center

- Both X and Y axes use identical independent controllers due to mechanical decoupling

Design procedure:

Step 1 — Define the combined plant including motor dynamics:

$$P(s) = 5.886 / [s^2 \cdot (0.0012s + 1)]$$

Step 2 — Apply MATLAB's pidtune function targeting the design specifications:

```
opts = pidtuneOptions('CrossoverFrequency', 1.5, 'PhaseMargin', 60)
C    = pidtune(P, 'PID', opts);
```

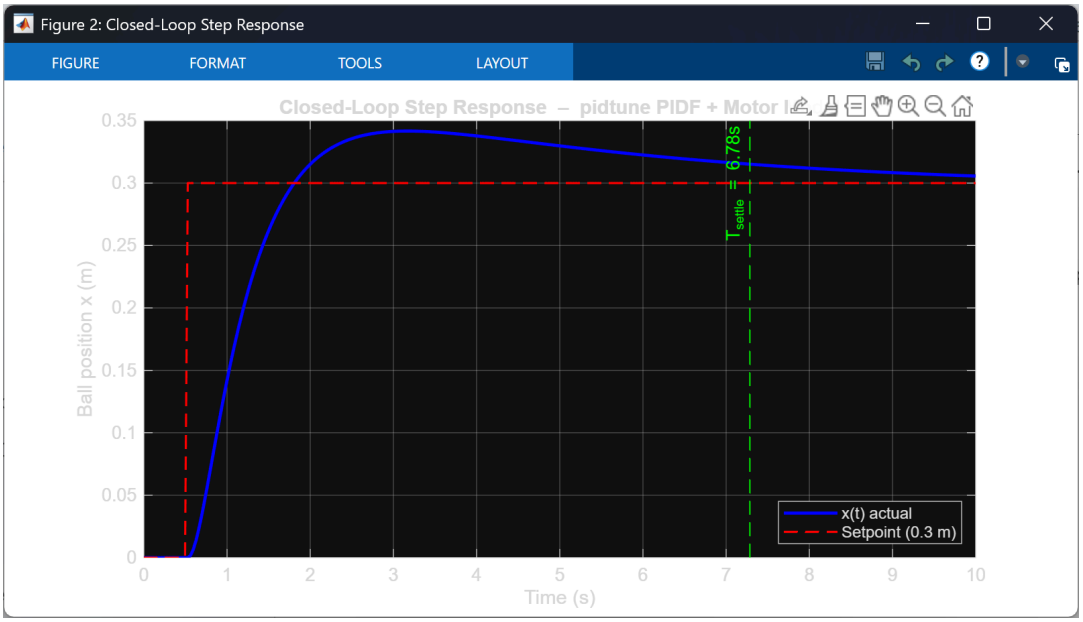
Step 3 — Extracted gains from pid tune output:

Gain	Value	Role
Kp	0.070303	Proportional — reacts to current error
Ki	0.004885	Integral — eliminates steady-state error
Kd	0.248320	Derivative — damps oscillation

Step 4 — Close the loop and verify stability margins and step response (shown in sections below).

3.2 Closed-Loop Simulations

The closed-loop system was simulated in MATLAB using the pidtune computed gains with a 33 ms sample time matching the STM32 hardware implementation.



Closed Loop Step Response

Simulation conditions:

- Setpoint: ball moves from center (0 m) to 0.3 m at t = 0.5 s
- Simulation duration: 10 seconds
- Sample time: 33 ms (from STM32 C code default dt = 0.033 s)
- Output saturation: ±8° (from STM32 C code: output_max = 8.0)

The step response shows the ball position tracking the 0.3 m setpoint successfully. The system is stable with the following performance metrics:

Metric	Value
Settling Time	6.78 s
Peak Overshoot	~13% (peak ≈ 0.34 m)
Steady-State Error	≈ 0 m (integral action eliminates offset)
Setpoint Reached	Yes — ball converges to 0.3 m

Stability Margins (from MATLAB margin function):

Margin	Achieved	Requirement	Status
Phase Margin	≥ 60°	> 45°	✓ Pass

Gain Margin	> 6 dB	> 6 dB	✓ Pass

The closed-loop pole-zero map confirms all poles are located in the left half plane, verifying closed-loop stability.

Note on Comparison with Hardware Implementation

The pidtune computed gains (Kp = 0.070303, Ki = 0.004885, Kd = 0.248320) represent the theoretically optimal values for the specified design targets. The hardware-tuned gains implemented on the STM32 (Kp = 0.06, Ki = 0.02051, Kd = 0.02165) were arrived at through iterative physical testing on the actual robot. The difference between the two sets of gains reflects real-world factors not captured in the mathematical model, including mechanical friction, camera latency, and discrete step resolution of the NEMA 17 stepper motors.

Programming

1. Computer Vision & GUI

The PC-side software is written in Python using OpenCV for real-time ball detection and Tkinter for the control GUI. It runs two concurrent threads — a CV loop processing camera frames at 30 FPS, and a GUI thread that lets the operator switch modes and tune parameters without interrupting tracking.

Computer Vision

Ball Detection Pipeline

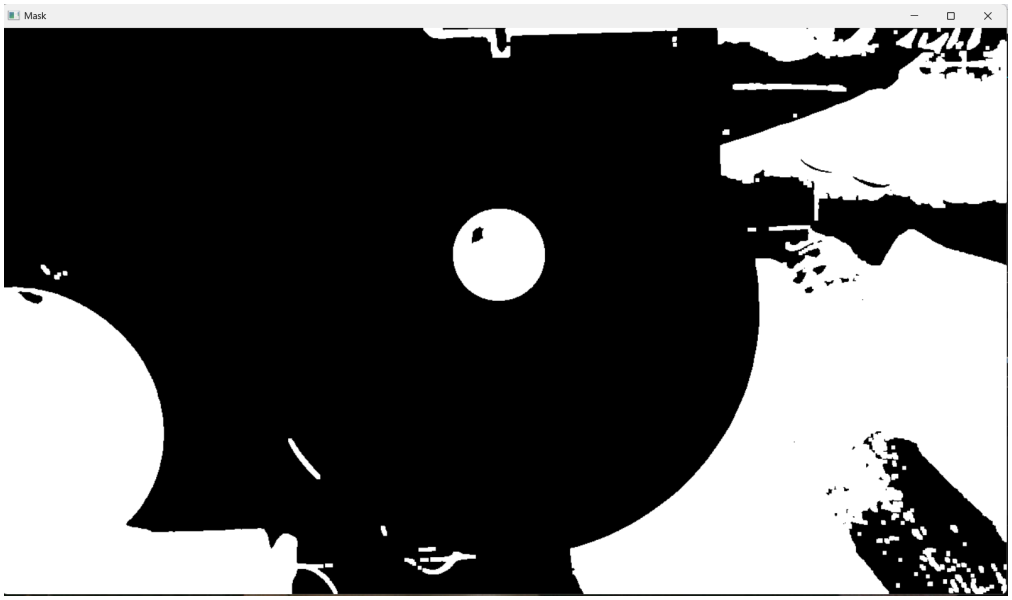
Each frame from the USB webcam goes through the following steps:

1. Pre-processing

The raw BGR frame is first blurred with a 5×5 Gaussian kernel to suppress noise, then converted to the HSV colour space. HSV is used instead of BGR because it separates colour (hue) from lighting intensity (value), making the detection robust to changes in ambient light. A colour threshold mask is applied for the black ball:

```
lower_ball = np.array([0,   0,   0])
upper_ball = np.array([180, 255, 50])
```

This isolates all pixels with very low brightness (value < 50) regardless of hue, which corresponds to the black ball. The mask is then cleaned up with one erosion pass (removes noise specks) followed by two dilation passes (fills gaps in the ball's silhouette).



Masking

2. Contour detection and validation

OpenCV's findContours extracts all connected regions from the binary mask. Each contour is then filtered through two criteria to reject false positives:

- **Radius bounds**: the minimum enclosing circle must be between a pre-set minimum and maximum radius in pixels (selected based on the physical ball size and the camera height above the platform). This helps reject false detections such as tiny noise blobs or large shadow regions that do not correspond to the ball.

  ```
  if  r < MIN_RADIUS or r > MAX_RADIUS:
          continue
  ```

- **Circularity check**: computed as $(4\pi \times \text{area}) / \text{perimeter}^2$, which equals 1.0 for a perfect circle. Only contours with circularity ≥ 0.75 are accepted — rejects non-circular objects like cables or plate edges.

  ```
  circ = (4 * np.pi * area) / (perim ** 2)
  if circ < CIRCULARITY_THRESHOLD:
          continue
  ```

3. Position extraction and filtering

After validating the contour, the largest valid contour is selected as the ball and its pixel position (ball_x, ball_y) is extracted. An exponential moving average (EMA filter) with α = 0.45 is then applied to reduce jitter from frame-to-frame noise:

```
x_smooth = 0.45 * ball_x + 0.55 * x_smooth
y_smooth = 0.45 * ball_y + 0.55 * y_smooth
```

The filtered position is then converted to millimetres relative to the frame centre using a calibrated mm per pixel scale factor (selected based on camera height from platform).

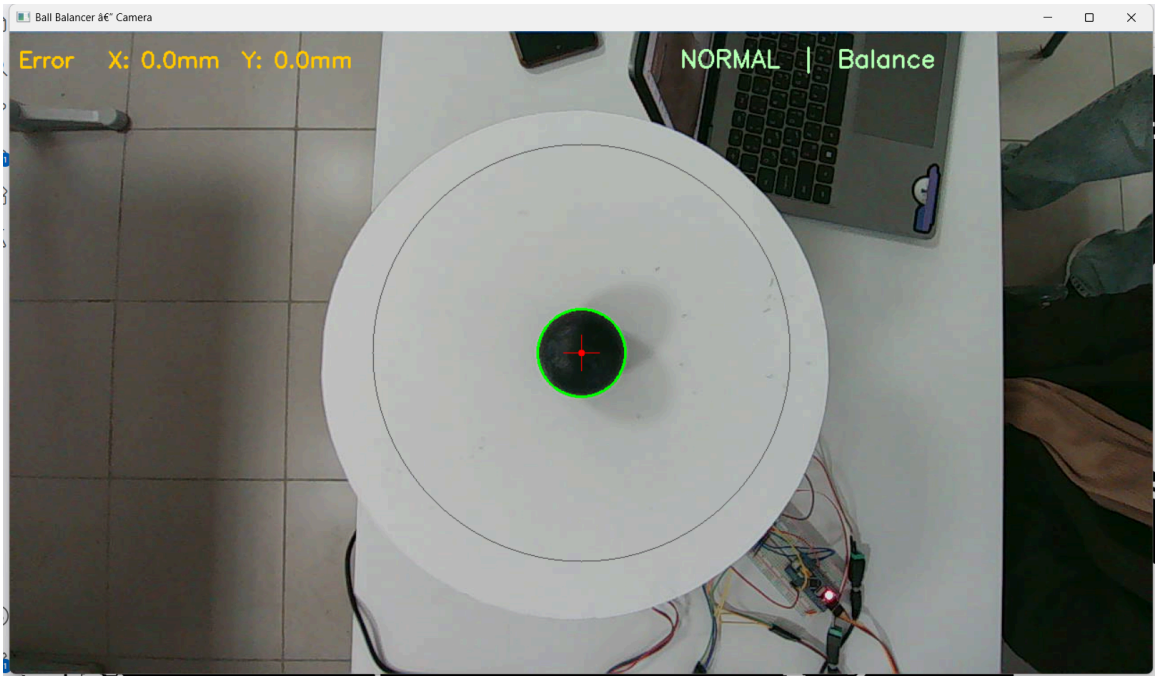
```
x_smooth_mm = (x_smooth - FRAME_CX) * MM_PER_PIXEL
y_smooth_mm = (y_smooth - FRAME_CY) * MM_PER_PIXEL
```

5. Error calculation and serial transmission

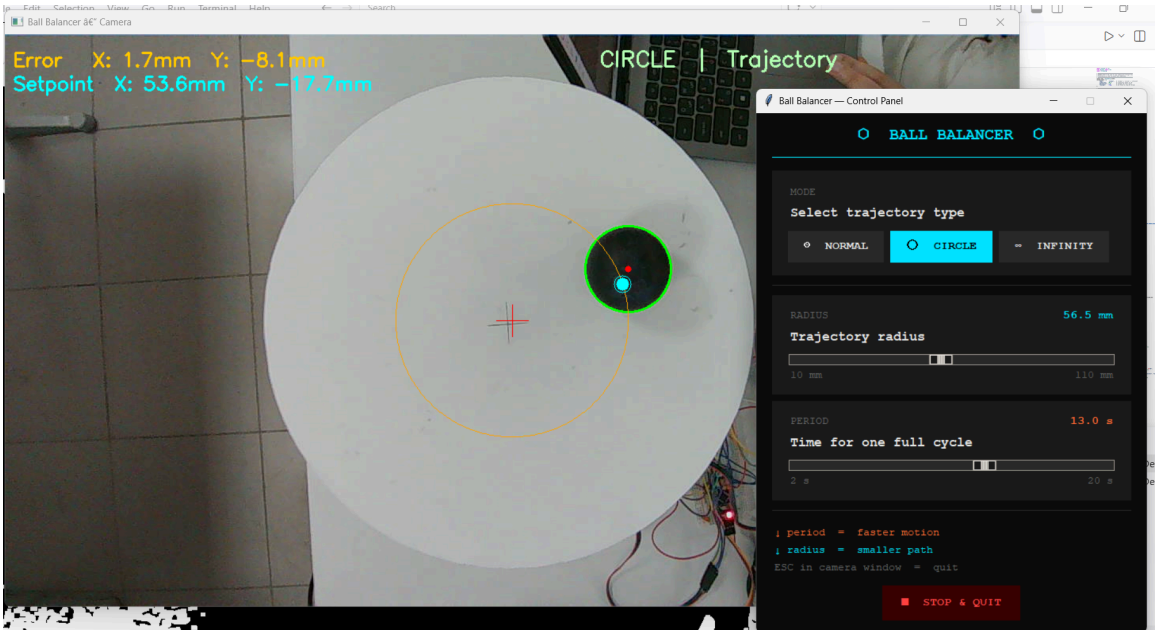
The smoothed ball coordinate position is compared against the current trajectory setpoint (centre for Normal mode, or a parametric path for Circle/Infinity/Sketch modes). The positional error in mm is multiplied by 10 to send as an integer over UART, avoiding floating point in the serial packet:

```
error_x_mm = x_smooth_mm - setpoint_x_mm
error_y_mm = y_smooth_mm - setpoint_y_mm
packet = f"<{int(error_x_mm * 10)},{int(error_y_mm * 10)}>\n"
stm_serial.write(packet.encode('utf-8'))
```

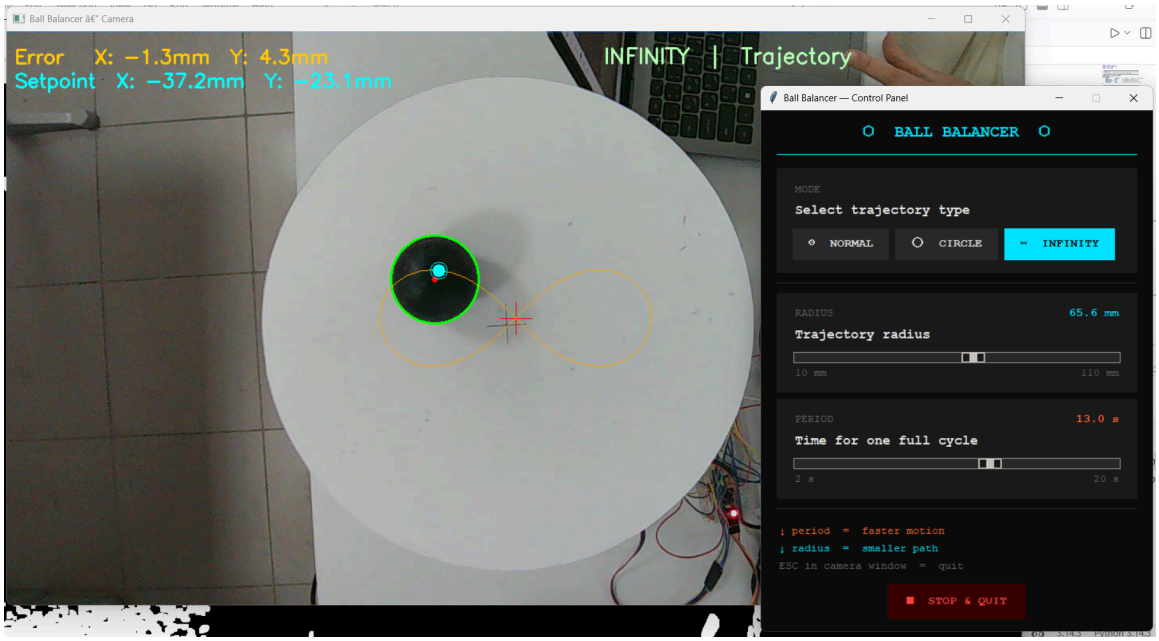
The STM32 receives this packet via UART DMA, parses the <x,y> format, and feeds the error directly into the PID controller.



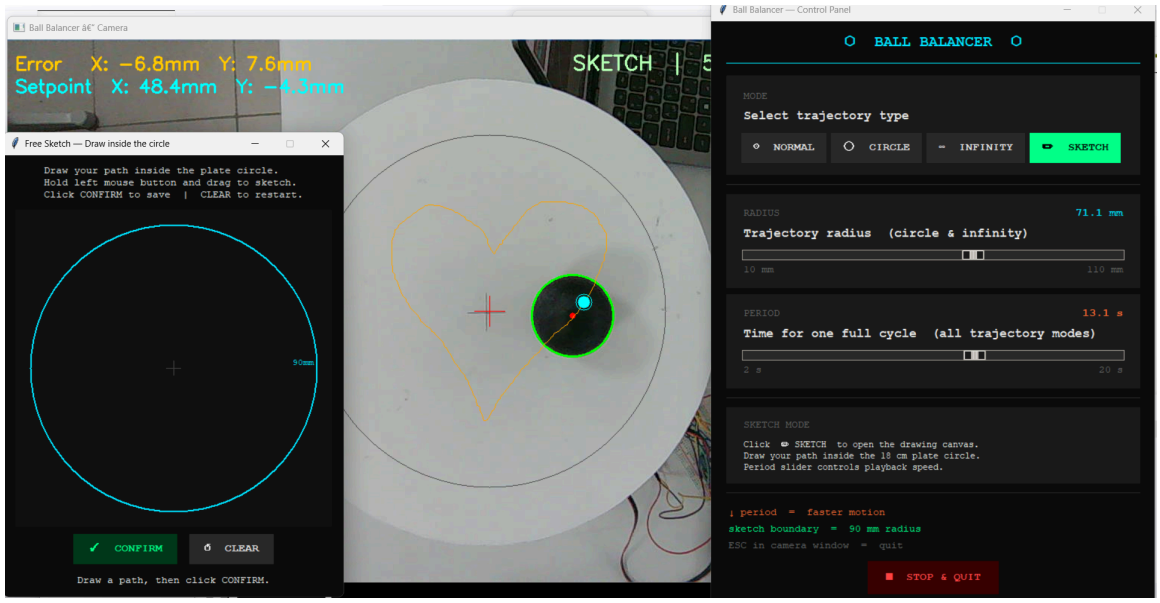
Center Balancing



Circle trajectory



Infinity Trajectory



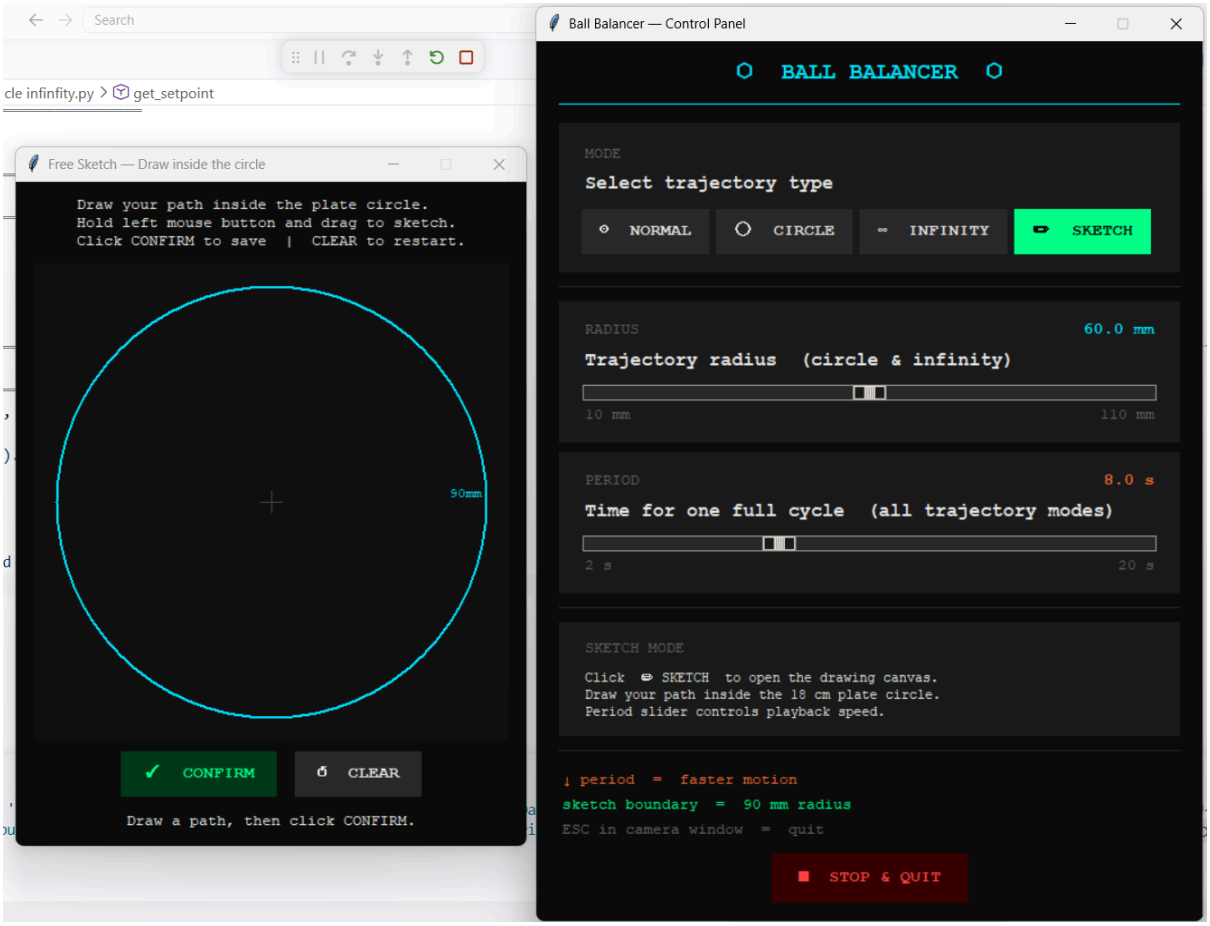
Free Sketch

GUI

The GUI exposes four modes selectable at runtime without restarting the system:

Mode	Setpoint behaviour
Normal	Fixed at (0, 0) — ball balances at centre
Circle	Parametric circle: $x = R \cdot \cos(\omega t)$, $y = R \cdot \sin(\omega t)$
Infinity	Lemniscate of Bernoulli (figure-of-eight curve)
Sketch	User draws a freehand path on a canvas; ball follows it as a closed loop

The radius (10–110 mm) and period (2–20 s) for the circle and infinity trajectories are adjustable via sliders in real time. In Sketch mode, a drawing canvas appears allowing the user to sketch the desired path. The drawing canvas enforces a 90 mm boundary to reject any out-of-bounds point too close to the plate edge.



GUI

STM32 Firmware (FreeRTOS)

The firmware runs on an STM32F103 at 72 MHz and is structured around FreeRTOS with two cooperative tasks. The PID computation and motor stepping are split into separate tasks that take turns running, while UART reception happens entirely in the background via DMA and an ISR.

System Clock & Timer

The MCU is clocked from an 8 MHz external crystal, multiplied by the PLL to 72 MHz. TIM2 is configured as a free-running microsecond counter:

```
TIM2->PSC = 71;          // 72 MHz ÷ 72 = 1 MHz → 1 tick per µs

TIM2->ARR = 0xFFFFFFFF;  // runs continuously, never resets

TIM2->CR1 |= TIM_CR1_CEN;
```

This 32-bit hardware timestamp is used for both measuring the actual time between PID updates (dt) and enforcing the minimum step interval between motor pulses, replacing all software delay loops for timing.

UART Reception (DMA + IDLE Interrupt)

USART1 runs at 115200 baud and receives packets from the PC in the format <x,y>. DMA Channel 5 is configured to transfer incoming bytes directly into rx_buffer in the background without any CPU involvement:

```
DMA1_Channel5->CPAR  = (uint32_t)&(USART1->DR);  // source: UART data register

DMA1_Channel5->CMAR  = (uint32_t)rx_buffer;      // destination: rx_buffer

DMA1_Channel5->CNDTR = RX_BUFFER_SIZE;           // up to 32 bytes
```

The UART IDLE line interrupt fires when the bus goes quiet after a complete packet. The ISR reads how many bytes DMA transferred, null-terminates the buffer, searches for the < start character, and parses the two integers:

```
uint16_t bytes_received = RX_BUFFER_SIZE - DMA1_Channel5->CNDTR;

sscanf(start, "<%d,%d>", &current_x, &current_y);

new_data_flag = 1;
```

DMA is then immediately reset to receive the next packet. The ISR only sets a plain volatile flag — no FreeRTOS API calls are made, keeping interrupt latency minimal.

PID Controller

Two independent PID controllers run on the X and Y axes with tuned gains:

```
pid->kp = 0.06f;

pid->ki = 0.02051f;

pid->kd = 0.02165f;
```

The derivative term applies a 70/30 low-pass filter to smooth out noise from numerical differentiation:

```
float raw_derivative = (error - pid->prev_error) / dt;

float derivative     = 0.7f * pid->prev_derivative + 0.3f * raw_derivative;
```

The integral is clamped to ±200 to prevent wind-up, and the output is clamped to ±8° — the physical tilt limit of the plate mechanism. The dt value is the actual elapsed time in seconds between consecutive received packets, measured from TIM2, so the controller adapts automatically if PC frame rate varies.

Stepper Motor Control

Each motor is driven by an A4988 driver over STEP and DIR GPIO pins on GPIOB. Update_Motors() is entirely non-blocking — it reads the current TIM2 timestamp and only pulses the STEP pin if enough time has elapsed since the last step:

```
if (diff != 0 &&

  (now - motor_x.last_step_time_us) >= motor_x.min_step_interval_us) {


   motor_x.step_port->BSRR = motor_x.step_pin_mask;  // STEP high

   delay_us(2);                                      // 2 µs pulse

   motor_x.step_port->BRR  = motor_x.step_pin_mask;  // STEP low
```

```
    motor_x.current_steps += (diff > 0) ? 1 : -1;
    motor_x.last_step_time_us = now;
}
```

The minimum step interval is 1200 µs, capping speed at ~833 steps/sec. The PID output angle is converted to a step target using 11.66 microsteps per degree of plate tilt (derived from the RRS inverse kinematics above). Direction is set from the sign of the remaining step error before each pulse.

FreeRTOS Task Structure

The firmware uses two tasks at equal priority (2), both calling taskYIELD() to cooperatively hand the CPU to each other:

vPIDTask (priority 2, 512-word stack) — polls new_data_flag on every iteration. When the flag is set by the ISR, it clears it, computes the actual dt, runs both PID controllers, and writes the new step targets to the motor structs. It then calls taskYIELD() to give the motor task a turn:

```
while (1) {
    if (new_data_flag == 1) {
        new_data_flag = 0;
        // ... compute dt, run PID, update motor targets
    }
    taskYIELD();
}
```

vMotorTask (priority 2, 512-word stack) — calls Update_Motors() on every iteration to check whether either motor needs a step pulse, then yields:

```
while (1) {
    Update_Motors();
```

```
    taskYIELD();
}
```

Because both tasks are at the same priority and both call taskYIELD(), the FreeRTOS scheduler alternates between them on every yield — effectively running PID check and motor step in a round-robin loop. This mirrors the original bare-metal while(1) structure but separates the two concerns into distinct tasks. Neither task ever blocks or sleeps, since Update_Motors() requires continuous polling of TIM2 to maintain microsecond step timing.